



Exercícios de JavaScript e DOM

15 exemplos resolvidos + 15 exercícios para o grupo
Só JavaScript

Como usar este caderno

São 15 exercícios em duplas: primeiro o **exemplo já resolvido**, para a gente ler e entender, e logo depois o **exercício do grupo**, que usa as mesmas coisas do exemplo com uma coisinha a mais ou diferente.

O caderno tem duas partes. Na parte 1 (A1 a A7) o código roda no console, sem página. Na parte 2 (B1 a B8) mexemos na página com o DOM, e por isso cada exercício diz quais elementos a página já tem (ids e classes) e o que deve acontecer.

Passo a passo sugerido

1. Leiam o exemplo resolvido em voz alta e digitem ele, sem copiar e colar.
2. Rodem e conferam se o resultado é igual ao que está escrito.
3. Leiam o “o que muda” do exercício e tentem fazer em grupo.
4. Só abram o PDF de respostas depois de tentar. Ele mostra HTML, CSS e JS e explica cada comando.

Nos exercícios de DOM

O HTML e o CSS não aparecem aqui, só o JS. Quem for testar precisa montar uma página com os elementos da tabela “A página já tem”. O PDF de respostas traz o HTML e o CSS prontos para copiar.

Parte 1 - Fundamentos no console

Aqui não tem página: abram o console do navegador (F12) ou um arquivo `script.js` simples e olhem a saída.

A1 - Console e variáveis

Revisar: `const`, `console.log`, texto com crases e `${ }`, conta simples • Apostila: capítulos 1, 3 e 4

Exemplo resolvido

Crie as variáveis `nome` e `idade` e mostre no console a frase “Olá, Ana! Você tem 15 anos.”

JS

```
const nome = "Ana";
const idade = 15;

console.log(`Olá, ${nome}! Você tem ${idade} anos.`);
```

SAÍDA NO CONSOLE

Olá, Ana! Você tem 15 anos.

Como pensar:

- Criamos `nome` com um texto (entre aspas).
- Criamos `idade` com um número (sem aspas).
- Montamos a frase com crases: o que está em `${ }` é trocado pelo valor da variável.
- `console.log` mostra o resultado no console.

Agora é com vocês

Faça o mesmo com o nome “Bruno” e a idade 16, e **acrescente uma segunda linha** que mostra em que ano a pessoa nasceu. Considere que o ano atual é 2026.

O que muda em relação ao exemplo

Além de guardar e mostrar valores, agora você **faz uma conta** com variáveis (ano atual menos a idade).

Resultado esperado no console:

SAÍDA NO CONSOLE

Olá, Bruno! Você tem 16 anos.
Você nasceu em 2010.

Escrevam o código aqui ou no computador:

A2 · Tipos e conversão

Revisar: texto x número, `typeof`, `Number()`, `toFixed`, `isNaN` • Apostila: capítulo 4

Exemplo resolvido

O texto "7" veio de um campo de formulário. Some 3 a ele do jeito errado e do jeito certo, e mostre o tipo de cada valor.

JS

```
const textoDoCampo = "7";

const errado = textoDoCampo + 3;
const certo = Number(textoDoCampo) + 3;

console.log(errado);
console.log(certo);
console.log(typeof textoDoCampo);
console.log(typeof certo);
```

SAÍDA NO CONSOLE

```
73
10
string
number
```

Como pensar:

- "7" está entre aspas: é **texto**, não número.
- Somar texto com número **junta** os dois: "7" + 3 vira "73".
- `Number("7")` converte para número, e aí a soma dá 10.
- `typeof` mostra o tipo: string (texto) ou number (número).

Agora é com vocês

As notas "9" e "6" chegaram como texto. Calcule a **média** delas e mostre com duas casas decimais ("Média: 7.50"). Depois, converta o texto "abc" para número e mostre se o resultado é inválido.

O que muda em relação ao exemplo

Além de converter, agora você **divide**, **formata** o número com `toFixed` e **testa** uma conversão que falha com `isNaN`.

Resultado esperado no console:

SAÍDA NO CONSOLE

```
Média: 7.50
true
```

Escrevam o código aqui ou no computador:

A3 · Decisões com if

Revisar: `if` / `else`, `else if`, `>=`, `&&` • Apostila: capítulo 5

Exemplo resolvido

Dada a variável `idade`, mostre “Pode entrar.” se for 18 ou mais, e “Entrada não permitida.” caso contrário.

JS

```
const idade = 17;

if (idade >= 18) {
  console.log("Pode entrar.");
} else {
  console.log("Entrada não permitida.");
}
```

SAÍDA NO CONSOLE

Entrada não permitida.

Como pensar:

- Guardamos a idade em uma variável.
- O `if` testa a condição `idade >= 18`.
- Como 17 não é maior ou igual a 18, a condição é falsa e roda o bloco do `else`.

Agora é com vocês

Dada a variável `nota = 5.5`, mostre “Aprovado” se for 6 ou mais, “Recuperação” se estiver entre 4 e 6 (4 incluído, 6 não), e “Reprovado” nos outros casos.

O que muda em relação ao exemplo

Agora são **três caminhos** (`else if`) e uma das condições junta duas comparações com `&&` (“e”).

Resultado esperado no console:

SAÍDA NO CONSOLE

Recuperação

Escrevam o código aqui ou no computador:

A4 - Repetições com for

Revisar: for, contador, % (resto), soma acumulada • Apostila: capítulo 6

Exemplo resolvido

Mostre no console os números de 1 a 5, no formato “Número 1”, “Número 2”...

JS

```
for (let i = 1; i <= 5; i++) {  
  console.log("Número " + i);  
}
```

SAÍDA NO CONSOLE

```
Número 1  
Número 2  
Número 3  
Número 4  
Número 5
```

Como pensar:

- O for tem três partes: onde começa (`i = 1`), até quando continua (`i <= 5`) e o que fazer a cada volta (`i++`).
- A cada volta o código entre `{ }` roda uma vez.
- No fim de cada volta, o `i` aumenta em 1.

Agora é com vocês

Percorra os números de 1 a 10. Mostre só os **pares**, no formato “2 é par”, e no final mostre a **soma dos pares**.

O que muda em relação ao exemplo

Agora existe um `if` **dentro** do laço (com `%`, o resto da divisão) e uma variável que **acumula** a soma.

Resultado esperado no console:

SAÍDA NO CONSOLE

```
2 é par  
4 é par  
6 é par  
8 é par  
10 é par  
Soma dos pares: 30
```

Escrevam o código aqui ou no computador:

A5 - Funções

Revisar: `function`, parâmetros, `return`, chamar uma função dentro da outra • Apostila: capítulo 7

Exemplo resolvido

Crie a função `dobro` que recebe um número e devolve o dobro dele. Teste com 4 e com 10.

JS

```
function dobro(numero) {  
  return numero * 2;  
}  
  
console.log(dobro(4));  
console.log(dobro(10));
```

SAÍDA NO CONSOLE

```
8  
20
```

Como pensar:

- A função recebe um valor no parâmetro `numero`.
- `return` devolve o resultado da conta.
- Cada chamada, como `dobro(4)`, usa a mesma receita com um valor diferente.

Agora é com vocês

Crie `precoComDesconto(preco, percentual)`, que devolve o preço depois do desconto, e `formatarReais(valor)`, que devolve o valor no formato “R\$ 180,00”. Mostre o preço final de um produto de 200 com 10% de desconto e de um de 80 com 25%.

O que muda em relação ao exemplo

Agora são **duas funções**, uma com **dois parâmetros**, e uma função é usada **dentro da chamada** da outra.

Resultado esperado no console:

SAÍDA NO CONSOLE

```
R$ 180,00  
R$ 60,00
```

Escrevam o código aqui ou no computador:

A6 · Listas (arrays)

Revisar: lista, filter, map, length, join • Apostila: capítulo 8

Exemplo resolvido

Dada a lista de notas [7, 4, 9, 5, 10], fique só com as notas 6 ou mais e mostre quais são e quantas são.

JS

```
const notas = [7, 4, 9, 5, 10];
const aprovadas = notas.filter(n => n >= 6);

console.log(aprovadas.join(", "));
console.log(aprovadas.length);
```

SAÍDA NO CONSOLE

```
7, 9, 10
3
```

Como pensar:

- A lista guarda cinco números em ordem.
- filter cria uma **lista nova** só com os itens em que a regra dá verdadeiro.
- join(", ") junta os itens em um texto para mostrar; length conta os itens.

Agora é com vocês

Dada a lista de preços [30, 80, 55, 120, 20], pegue só os preços **acima de 50** e depois aplique **10% de desconto** em cada um. Mostre os preços acima de 50, os preços com desconto e quantos são.

O que muda em relação ao exemplo

Agora você usa filter e map em sequência: um escolhe os itens, o outro transforma cada um.

Resultado esperado no console:

SAÍDA NO CONSOLE

```
80, 55, 120
72, 49.5, 108
3
```

Escrevam o código aqui ou no computador:

A7 · Objetos e lista de objetos

Revisar: objeto, objeto.campo, for...of, find • Apostila: capítulo 8

Exemplo resolvido

Crie um objeto `aluno` com `nome` e `nota`. Mostre o nome, depois mude a nota para 9 e mostre a nota.

JS

```
const aluno = { nome: "Ana", nota: 8 };

console.log(aluno.nome);
aluno.nota = 9;
console.log(aluno.nota);
```

SAÍDA NO CONSOLE

```
Ana
9
```

Como pensar:

- O objeto é uma “ficha” com campos: `nome` e `nota`.
- `aluno.nome` lê um campo.
- `aluno.nota = 9` altera o campo (permitido mesmo com `const`: o que não pode é trocar o objeto inteiro).

Agora é com vocês

Crie uma **lista** com três alunos (Ana nota 8, Bruno nota 6, Carla nota 10). Mostre cada aluno no formato “Ana: 8”, depois a **média da turma** e, por fim, a nota da Carla (procure-a pelo nome).

O que muda em relação ao exemplo

Agora você tem uma **lista de objetos**, percorre com `for...of`, calcula a **média** e procura um item com `find`.

Resultado esperado no console:

SAÍDA NO CONSOLE

```
Ana: 8
Bruno: 6
Carla: 10
Média da turma: 8
Nota da Carla: 10
```

Escrevam o código aqui ou no computador:

Parte 2 - DOM e página

Cada exercício parte de uma página que já existe. A tabela mostra o que ela tem, e vocês escrevem o JS.

B1 - Selecionar e mudar texto

Revisar: `getElementById`, `addEventListener`, `textContent`, `classList.toggle`, `let` • Apostila: capítulos 9 a 11

Exemplo resolvido

Ao clicar no botão, o parágrafo deve passar a mostrar “Você clicou!”.

A página já tem:

Seletor	O que é
#botao	um botão com o texto “Clique aqui”
#mensagem	um parágrafo que começa com “Ainda não clicou.”

O que deve acontecer:

- Ao abrir: o parágrafo mostra “Ainda não clicou.”
- Depois de clicar: o parágrafo mostra “Você clicou!”.

JS

```
const botao = document.getElementById("botao");
const mensagem = document.getElementById("mensagem");

botao.addEventListener("click", function () {
  mensagem.textContent = "Você clicou!";
});
```

Como pensar:

- O HTML tem um botão e um parágrafo, cada um com um `id`.
- O JavaScript pega os dois elementos pelo `id`.
- Ele fica “ouvindo” o clique no botão.
- Quando o clique acontece, a função troca o texto do parágrafo.

Agora é com vocês

Crie um contador de cliques. Dois botões: “+1” soma um clique e “Zerar” volta a zero. O texto deve mostrar “Cliques: N”. Quando chegar a 5 ou mais, o texto deve ficar vermelho (use uma classe).

O que muda em relação ao exemplo

Agora existe uma **variável que muda** (`let`), **dois botões**, uma função que **atualiza a tela** e uma classe que é **ligada ou desligada conforme uma condição**.

A página já tem:

Seletor	O que é
#contador	um parágrafo com o texto “Cliques: 0”
#mais	botão “+1”
#zerar	botão “Zerar”
.alerta	classe do CSS que deixa o texto vermelho

O que deve acontecer:

- Cada clique em “+1” aumenta o número.
- A partir de 5, o texto fica vermelho e em negrito.
- “Zerar” volta para “Cliques: 0” e tira o vermelho.

Escrevam o código aqui ou no computador:

B2 - Classes e localStorage (modo escuro)

Revisar: `document.body`, `classList.toggle` / `contains`, `localStorage`, ternário • Apostila: capítulos 11 e 18

Exemplo resolvido

Ao clicar no botão, a página deve alternar entre o visual claro e o escuro. Faça isso ligando e desligando uma classe no `<body>`.

A página já tem:

Seletor	O que é
<code>#botao-modo</code>	um botão com o texto “Modo escuro”
<code>body.escuro</code>	regra do CSS que pinta a página de escuro

O que deve acontecer:

- Primeiro clique: a página fica escura.
- Segundo clique: volta ao claro.

JS

```
const botao = document.getElementById("botao-modo");

botao.addEventListener("click", function () {
  document.body.classList.toggle("escuro");
});
```

Como pensar:

- O CSS já sabe como a página fica escura: é só a classe `escuro` no `body`.
- O JavaScript não pinta nada: ele só liga e desliga essa classe.
- A cada clique, `toggle` inverte: se tinha a classe, tira; se não tinha, põe.

Agora é com vocês

Melhore o modo escuro: o texto do botão deve mudar (“Modo claro” quando estiver escuro e “Modo escuro” quando estiver claro) e a escolha deve ser **lembrada** quando a pessoa recarregar a página.

O que muda em relação ao exemplo

Agora o JS também **muda o texto do botão, guarda a escolha** no `localStorage` e **lê** essa escolha ao abrir a página.

A página já tem:

Seletor	O que é
<code>#botao-modo</code>	um botão (o texto muda pelo JS)
<code>body.escuro</code>	regra do CSS que pinta a página de escuro

O que deve acontecer:

- O texto do botão alterna entre “Modo escuro” e “Modo claro”.
- Depois de recarregar a página, o modo escolhido continua.

Escrevam o código aqui ou no computador:

B3 - Criar elementos com JavaScript

Revisar: `createElement`, `append`, `for...of`, função que devolve um elemento, `sort` • Apostila: capítulo 12

Exemplo resolvido

Dada a lista `["Asiático", "Medieval", "Cyberpunk"]`, crie com JavaScript um item de lista (`<li>`) para cada nome, dentro de uma lista que já existe no HTML.

A página já tem:

Seletor	O que é
<code>#lista</code>	uma lista <code></code> vazia

O que deve acontecer:

- A página mostra uma lista com os três nomes, um por linha.

JS

```
const saloes = ["Asiático", "Medieval", "Cyberpunk"];
const lista = document.getElementById("lista");

for (const nome of saloes) {
  const item = document.createElement("li");
  item.textContent = nome;
  lista.append(item);
}
```

Como pensar:

- O HTML tem só a lista vazia.
- O laço passa por cada nome da lista de salões.
- Para cada nome, cria um `<li>`, coloca o texto e encaixa na lista.
- Só quando o `append` acontece o item aparece na tela.

Agora é com vocês

Dada uma lista de lâmpadas (autor e votos), mostre uma **carta** para cada uma, **da mais votada para a menos votada**. A carta mostra o autor em negrito e a quantidade de votos. As cartas com 10 votos ou mais devem ter um destaque (use uma classe).

O que muda em relação ao exemplo

Agora o elemento é montado dentro de uma **função que devolve a carta**, com **elementos dentro de elementos**, uma **classe condicional** e uma **lista ordenada** antes de criar.

A página já tem:

Seletor	O que é
<code>#palco</code>	uma <code><div></code> vazia
<code>.carta</code>	classe do CSS para cada carta
<code>.carta.destaque</code>	classe do CSS para as cartas com 10 votos ou mais

O que deve acontecer:

- Aparecem 3 cartas na ordem: theo (30), luna (12), maya (5).
- As cartas de theo e luna têm o fundo amarelo e a borda dourada.

Escrevam o código aqui ou no computador:

B4 · Eventos e delegação

Revisar: `addEventListener`, `evento.target`, `closest`, `dataset`, `find` • Apostila: capítulo 13

Exemplo resolvido

Há uma lista com três salões. Ao clicar em um deles, mostre abaixo “Você escolheu: [nome do salão]”. Use **um único ouvinte**, colocado na lista.

A página já tem:

Seletor	O que é
#lista	uma <code></code> com três <code></code> : Asiático, Medieval e Cyberpunk
#escolhido	um parágrafo com o texto “Nenhum salão escolhido.”

O que deve acontecer:

- Ao clicar em “Medieval”: aparece “Você escolheu: Medieval”.

JS

```
const lista = document.getElementById("lista");
const escolhido = document.getElementById("escolhido");

lista.addEventListener("click", function (evento) {
  escolhido.textContent = "Você escolheu: " + evento.target.textContent;
});
```

Como pensar:

- Em vez de um ouvinte por item, colocamos **um só** na lista (o elemento pai).
- Clicar em um `<li>` também “chega” na lista: o evento sobe.
- `evento.target` diz qual item foi clicado de verdade.
- Mostramos o texto desse item no parágrafo.

Agora é com vocês

Mostre uma carta para cada lâmpada (autor, votos e um botão “+1”). Ao clicar no “+1” de uma carta, **só os votos dela** aumentam, tanto na tela quanto na lista de dados. Use **um único ouvinte** no palco, e clicar fora do botão não deve fazer nada.

O que muda em relação ao exemplo

Agora são **vários botões** gerados pelo JS, o ouvinte usa `closest` para achar o botão e a carta, e o `id` da lâmpada é guardado em `data-id` para atualizar o dado certo.

A página já tem:

Seletor	O que é
#palco	uma <code><div></code> vazia
.carta	classe do CSS para cada carta

O que deve acontecer:

- Cada carta mostra “autor”, “N votos” e um botão “+1”.
- Clicar em “+1” aumenta só aquela carta.
- Clicar no espaço da carta (fora do botão) não muda nada.

Escrevam o código aqui ou no computador:

B5 · Formulários e validação

Revisar: evento `submit`, `preventDefault`, `value`, `trim`, `length`, função que devolve o erro • Apostila: capítulo 14

Exemplo resolvido

Faça um formulário com o campo “apelido”. Ao enviar, se o apelido (sem espaços nas pontas) tiver menos de 3 letras, mostre “O apelido precisa ter pelo menos 3 letras.”. Se não, mostre “Bem-vindo, [apelido]!”.

A página já tem:

Seletor	O que é
#form	um formulário
#apelido	um campo de texto dentro do formulário
#retorno	um parágrafo vazio para a mensagem

O que deve acontecer:

- Digitando “ab” e enviando: aparece a mensagem de erro.
- Digitando “luna” e enviando: aparece “Bem-vindo, luna!”.
- A página não recarrega.

JS

```
const form = document.getElementById("form");
const apelido = document.getElementById("apelido");
const retorno = document.getElementById("retorno");

form.addEventListener("submit", function (evento) {
  evento.preventDefault();

  const valor = apelido.value.trim();
  if (valor.length < 3) {
    retorno.textContent = "O apelido precisa ter pelo menos 3 letras.";
  } else {
    retorno.textContent = "Bem-vindo, " + valor + "!";
  }
});
```

Como pensar:

- O formulário avisa o JS quando é enviado (evento `submit`).
- `preventDefault` evita que a página recarregue.
- Lemos o que foi digitado e tiramos espaços das pontas.
- Um `if` confere o tamanho e escolhe qual mensagem mostrar.

Agora é com vocês

Faça um formulário de cadastro com apelido (mínimo 3 letras), senha (mínimo 6 caracteres) e repetição da senha. Mostre **uma mensagem de erro diferente para cada problema**, em vermelho, e “Conta criada!” em verde quando estiver tudo certo. Mostre também, enquanto a pessoa digita, quantas letras o apelido tem.

O que muda em relação ao exemplo

Agora são **três campos**, uma **função** `validar` que só confere e devolve o erro, o evento `input` (a cada letra) e uma **classe que muda a cor** da mensagem.

A página já tem:

Seletor	O que é
#form	um formulário
#apelido	campo de texto
#contador	um <code><small></code> com “0 letras”
#senha	campo de senha
#confirma	campo de senha (repetição)
#retorno	um parágrafo vazio
.erro / .ok	classes do CSS para vermelho e verde

O que deve acontecer:

- Ao digitar no apelido, o contador mostra “N letras”.
- Cada erro (apelido curto, senha curta, senhas diferentes) tem sua mensagem em vermelho.
- Tudo certo: “Conta criada!” em verde.

Escrevam o código aqui ou no computador:

B6 - Janela que abre e fecha

Revisar: classe que esconde (oculto), `classList.add / remove`, `dataset`, tecla Esc, clique no fundo •
Apostila: capítulo 27

Exemplo resolvido

Faça um botão “Abrir janela” que mostra uma janela por cima da página, e um botão “Fechar” dentro dela que a esconde.

A página já tem:

Seletor	O que é
#abrir	botão “Abrir janela”
#janela	uma <div> que cobre a tela, já começa com a classe <code>oculto</code>
#fechar	botão “Fechar” dentro da janela
.oculto	classe do CSS que esconde (<code>display: none</code>)

O que deve acontecer:

- Ao abrir a página, a janela não aparece.
- Clicar em “Abrir janela” mostra a janela por cima, com fundo escurecido.
- Clicar em “Fechar” esconde de novo.

JS

```
const janela = document.getElementById("janela");

document.getElementById("abrir").addEventListener("click", function () {
  janela.classList.remove("oculto");
});

document.getElementById("fechar").addEventListener("click", function () {
  janela.classList.add("oculto");
});
```

Como pensar:

- A janela existe no HTML, mas começa com a classe `oculto`, que a esconde.
- “Abrir” **tira** essa classe, e a janela aparece.
- “Fechar” **põe** a classe de volta, e a janela some.

Agora é com vocês

Faça uma lista de duas cartas. Ao clicar em uma carta, a **mesma janela** deve abrir mostrando a mensagem e o autor **daquela** carta. A janela deve fechar de **três jeitos**: pelo botão X, clicando no fundo escuro e apertando a tecla Esc.

O que muda em relação ao exemplo

Agora **uma só janela** serve para várias cartas (o JS **preenche** os textos), os dados vêm de `data-*`, e há **três formas de fechar** (botão, fundo e tecla).

A página já tem:

Seletor	O que é
#palco	uma <div> com duas <div class="carta">, cada uma com <code>data-autor</code> e <code>data-mensagem</code>
#janela	a janela, com a classe <code>oculto</code>
#fechar	botão X dentro da janela
#janela-mensagem	parágrafo vazio dentro da janela

Seletor	O que é
#janela-autor	um <code></code> vazio dentro da janela

O que deve acontecer:

- Clicar na carta da luna abre a janela com a mensagem e o autor dela.
- Clicar na carta do theo mostra os dados dele na mesma janela.
- X, clique no fundo escuro e Esc fecham; clicar dentro da caixa branca não fecha.

Escrevam o código aqui ou no computador:

B7 - Temporizador e datas

Revisar: `setInterval`, `clearInterval`, `Date.now`, milissegundos, `Math.floor`, % • Apostila: capítulos 15 e 16

Exemplo resolvido

Faça uma contagem regressiva de 10 até 1 na tela, diminuindo um número a cada segundo. Quando passar do 1, mostre “Acabou!” e pare o relógio.

A página já tem:

Seletor	O que é
#tempo	um parágrafo com o texto “10”

O que deve acontecer:

- Mostra 10, depois 9, 8... a cada segundo.
- Depois do 1, aparece “Acabou!” e a contagem para.

JS

```
const tempo = document.getElementById("tempo");
let restante = 10;

function tick() {
  restante--;
  if (restante <= 0) {
    tempo.textContent = "Acabou!";
    clearInterval(relogio);
  } else {
    tempo.textContent = restante;
  }
}

const relogio = setInterval(tick, 1000);
```

Como pensar:

- `restante` guarda os segundos que faltam.
- `setInterval` chama a função `tick` a cada 1 segundo.
- A cada chamada, diminuimos 1 e atualizamos o texto.
- Quando chega a zero, escrevemos “Acabou!” e paramos o relógio.

Agora é com vocês

Mostre quanto tempo falta até uma data futura no formato “Xd Xh Xm Xs”, atualizando a cada segundo. Depois, faça um botão que **pausa** e **retoma** a contagem (o texto do botão alterna entre “Pausar” e “Retomar”). Quando a data já tiver passado, mostre “Liberada!”.

O que muda em relação ao exemplo

Agora o tempo vem de uma **conta com datas** (data alvo menos agora, em milissegundos), é **formatado** com divisões e %, e o relógio pode ser **parado e criado de novo** (`relogio = null` indica que está parado).

A página já tem:

Seletor	O que é
#tempo	um parágrafo vazio
#pausar	um botão

O que deve acontecer:

- Mostra algo como “1180d 5h 12m 30s”, mudando a cada segundo.
- “Pausar” congela a contagem e vira “Retomar”.

- “Retomar” volta a contar (já com o tempo certo).

Escrevam o código aqui ou no computador:

B8 - Várias páginas em um HTML

Revisar: `querySelectorAll`, `forEach`, `hashchange`, `location.hash`, `slice`, objeto de páginas • Apostila: capítulos 22 a 25

Exemplo resolvido

Monte um site de **duas páginas** (Início e Sobre) dentro de **um único HTML**. Dois links no topo trocam a página mostrada, usando o `#` do endereço.

A página já tem:

Seletor	O que é
<code>nav</code>	dois links: <code>href="#inicio"</code> e <code>href="#sobre"</code>
<code>#inicio</code>	uma <code><section class="pagina"></code>
<code>#sobre</code>	outra <code><section class="pagina"></code>
<code>.pagina / .pagina.ativa</code>	classes do CSS: esconde todas e mostra só a ativa

O que deve acontecer:

- Ao abrir, aparece a página Início.
- Clicar em “Sobre” mostra só a página Sobre.
- O botão voltar do navegador volta para Início.

JS

```
function mostrarPagina(nome) {
  document.querySelectorAll(".pagina").forEach(function (secao) {
    secao.classList.remove("ativa");
  });
  document.getElementById(nome).classList.add("ativa");
}

window.addEventListener("hashchange", function () {
  mostrarPagina(location.hash.slice(1));
});

mostrarPagina("inicio");
```

Como pensar:

- O CSS esconde todas as páginas e só mostra a que tem a classe `ativa`.
- `mostrarPagina` tira a classe de todas e liga só na escolhida.
- Clicar em um link `href="#sobre"` muda o `#` do endereço e dispara `hashchange`.
- Nesse evento, lemos o `#` e mostramos a página certa.

Agora é com vocês

Amplie o site para **três páginas** (Início, Salão e Sobre). Ao trocar de página, o **link do menu atual** deve ficar destacado, o **título da aba** deve mudar e, se o endereço for inventado (como `#xyz`) ou estiver vazio, o site deve mostrar o Início.

O que muda em relação ao exemplo

Agora existe um **objeto** que liga cada nome à sua seção e ao seu título, o menu usa `data-pagina` e `toggle` com condição, o `document.title` muda e há um **plano B** para endereços desconhecidos.

A página já tem:

Seletor	O que é
#menu	um <nav> com três links (href="#inicio", #salao, #sobre), cada um com data-pagina
#pagina-inicio, #pagina-salao, #pagina-sobre	três <section class="pagina">
.pagina / .pagina.ativa	classes do CSS
#menu a.ativo	classe do CSS para o link da página atual

O que deve acontecer:

- Ao abrir sem #: mostra o Início, com o link “Início” em destaque e título “Meu site | Início”.
- Clicar em “Salão”: mostra o Salão, destaque no link “Salão”, título “Meu site | Salão”.
- Endereço #xyz: volta para o Início.

Escrevam o código aqui ou no computador: